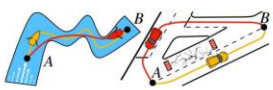
One Week Ago

1. [**Learnable Omega**] Brainstorm and implement architecture to learn drift vector omega
   - Implement some fix such that the model is more responsive to alignment loss
2. [**Visualisation**]
   - We could plot pairs of images that have the highest magnitude of omega, and pairs with the smallest magnitude
   - Reduce the IDs in the query so that it is more manageable (perhaps only 10-15)
   - Find a way to keep these IDs always the same across runs (just for the visualization), and not dependent on how the dataset is shuffled given the seed
3. [**Dataset**] Integrate cross-view dataset AG-ReID v2

Problem Motivation: Why Asymmetry in Re-ID?



Proposition: Standard Riemannian geometry enforces symmetry ($d(A, B) = d(B, A)$), but real data live in complex asymmetrical manifolds, where $d(A, B) \neq d(B, A)$.

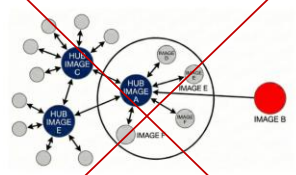
Cross-Res Asymmetry

A B

Camera Viewpoint Bias

A B

Asymmetrical Re-Ranking



Euclidean distance **is symmetric**. k-Reciprocal Nearest Neighbor is **not**.

Cross-Resolution Asymmetry:
Matching pictures from query and gallery sets of different resolution depends on the direction.
LR -> HR: The model must *hallucinate* or super-resolve missing details (high uncertainty). The "path" involves inventing plausible high-frequency data.
HR -> LR: The model simply needs to downsample or degrade the features (low uncertainty). The "path" involves discarding existing data.
The conditional probability of a match between two images depends on which set is the gallery and which set is the query

- Estimating the resolution of the images
- Manually say that
    - If the resolution is smaller, change the distance w.r.t. the resolution
    - Euclidean distance + non-positive term that matches the difference in resolution
    - I'm only going to look at a small subset, different resolutions, adding different terms
    - Is it a good modifications

Camera Viewpoint Bias:
A -> B: match a face and chest to a back and shoulder; features are disjoint
Has access to unique features of the person's identity
Uses these to preserve invariant features (e.g., global color histogram, hair color)
Can form a strong confident representation of the feature and use it to query

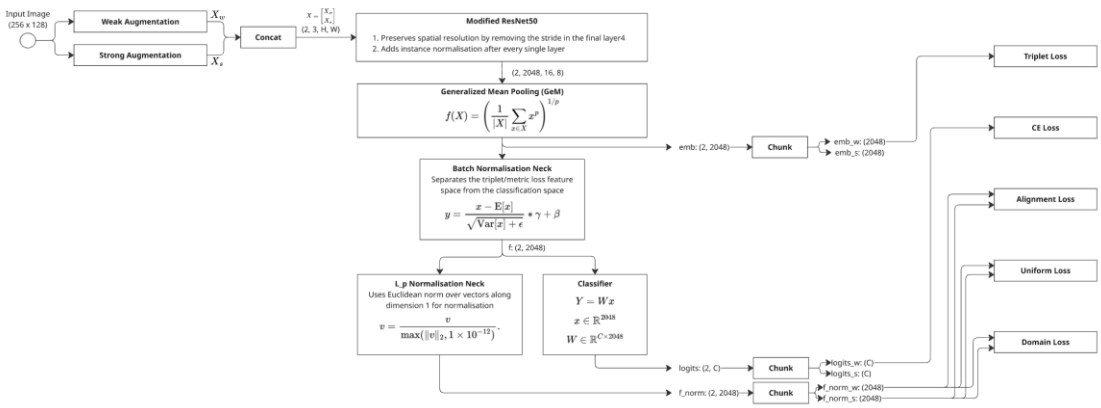
B -> A: Relies on different features due to the "Distractor"
Image is corrupted by "Occluder" -> "Blue" signal is weaker and mixed with "Black Suit" noise
Model must first filter out the noise using attention
Might rely on inferring the face from head shape and hair color

Creates a directed graph structure in the manifold of "person appearance"
Shortest distance between two images depends on the starting point

Asymmetrical Re-Ranking:
In high-dimensional spaces, certain "hub" data points appear in k-nearest neighbors of a large number of other points
If image A is a hub, it might be the nearest neighbor of image B. but Image B is not in the nearest neighbor of Image A.
Many Re-ID models use k-Reciprocal Nearest Neighbor relationships in their re-ranking methods -> introduces asymmetry

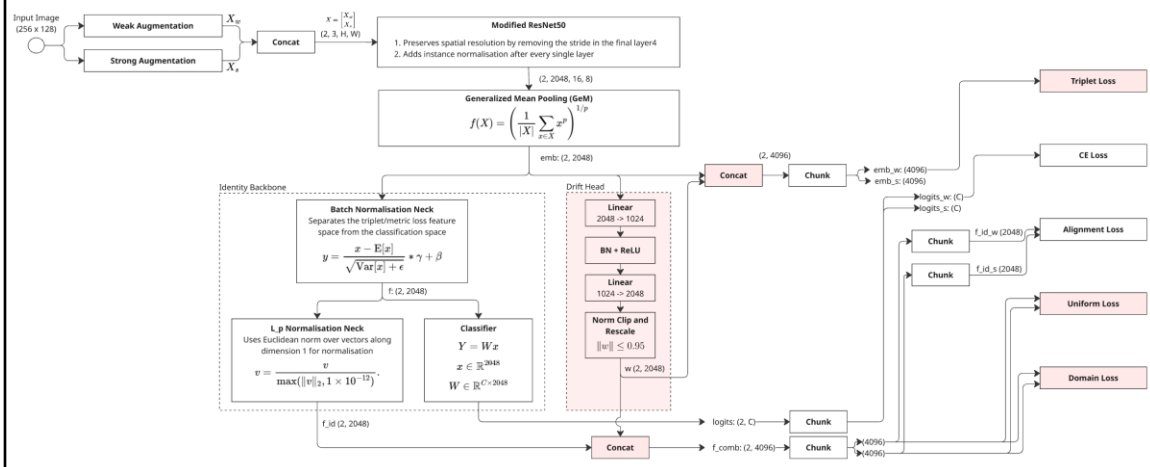
- Reciprocal kNN is a symmetrical version of kNN

Recap: Original BAU



Foundation: This is our starting point—the standard "Balancing Alignment and Uniformity" architecture.
Components: It uses a ResNet50 backbone with Generalized Mean Pooling (GeM) and a Batch Normalization neck.
The Four Objectives: The model is optimized using Cross-Entropy (classification), Triplet Loss (ranking), Alignment Loss (invariance), and Uniformity/Domain Losses (feature distribution).
Limitations: This baseline operates purely in Euclidean space, forcing symmetry on asymmetric data.

Learnable Omega – Architecture Introduction



The Split: We introduce a dedicated "Drift Branch" alongside the standard identity branch. The model now outputs two vectors: an **Identity Feature** (Who is this?) and a **Drift Vector** (What is the viewpoint bias?).

Gradient Flow:

- The **Classifier** (Cross-Entropy) sees *only* the identity features. We do not want the drift (viewpoint noise) to confuse the identity classification.
- The **Drift Head** receives pre-BN features to preserve magnitude information, which is crucial for estimating the "intensity" of the viewpoint distortion.

- Sigmoid / Exponential
- Stable behavior should be zero

- Norm to zero ->

Learnable Omega – Model Introduction

Drift Branch

$$d_F(\mathbf{x}, \mathbf{y}) = \underbrace{\|\mathbf{x}_{\text{id}} - \mathbf{y}_{\text{id}}\|_2}_{\text{identity distance}} + \underbrace{\mathbf{x}_\omega \cdot (\mathbf{y}_{\text{id}} - \mathbf{x}_{\text{id}})}_{\text{viewpoint-dependent drift}}$$

Let *p* and *q* be the dimension for the identity feature and the drift vector, respectively.
Given a vector embedding $\mathbf{x} \in \mathbb{R}^{p+q}$

$\mathbf{x}_{\text{id}} \in \mathbb{R}^p$ Identity Feature *"Who is this?"*	$\mathbf{x}_\omega \in \mathbb{R}^q$ Drift Vector *"E.g., What Viewpoint?"*

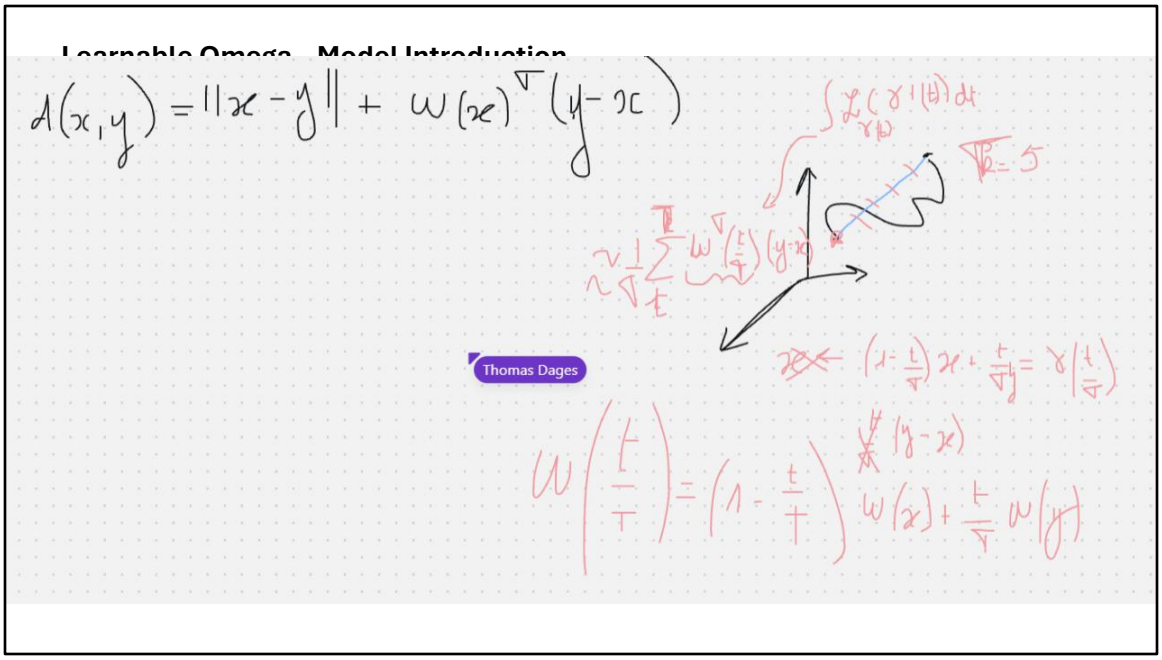
- $\mathbf{x}_\omega$ is norm-constrained: $\|\mathbf{x}_\omega\| \leq 0.95$
- $d_F(\mathbf{x}, \mathbf{y}) \neq d_F(\mathbf{y}, \mathbf{x})$: asymmetric ranking

Mathematical Definition: We replace Euclidean distance with the Finsler-Randers distance: $d_F(x,y) = ||x_{id} - y_{id}||_2 + \omega_x \cdot (y_{id} - x_{id})$.
Interpretation:
- The first term is the standard symmetric distance.
- The second term is the **asymmetric drift**, representing the "wind" or "current" of the viewpoint.

Constraints: The drift vector ω is norm-constrained ($||\omega|| \leq 0.95$) to ensure the distance remains positive definite (the "Randers condition").

This is cheat -> it does not calculate the geodesic

- Approximates a length of the straight line
- Integral of the term would be a better approximation



Mathematical Definition: We replace Euclidean distance with the Finsler-Randers distance: $d_F(x,y) = ||x_{id} - y_{id}||_2 + \omega_x \cdot (y_{id} - x_{id})$.
Interpretation:

- The first term is the standard symmetric distance.
- The second term is the **asymmetric drift**, representing the "wind" or "current" of the viewpoint.

Constraints: The drift vector ω is norm-constrained ($||\omega|| \leq 0.95$) to ensure the distance remains positive definite (the "Randers condition").

This is cheat -> it does not calculate the geodesic

- Approximates a length of the straight line
- Integral of the term would be a better approximation

Learnable Omega – Loss Functions

$$\mathbf{f} = \begin{bmatrix} \mathbf{f}_{\text{id}} \\ \mathbf{f}_{\omega} \end{bmatrix} \in \mathbb{R}^{p+q}$$

$$d_F(\mathbf{x}, \mathbf{y}) = \underbrace{\|\mathbf{x}_{\text{id}} - \mathbf{y}_{\text{id}}\|_2}_{\text{identity distance}} + \underbrace{\mathbf{x}_{\omega} \cdot (\mathbf{y}_{\text{id}} - \mathbf{x}_{\text{id}})}_{\text{viewpoint-dependent drift}}$$

$$\mathcal{L}_{\text{align}} = \sum_{(i,j) \in \mathcal{I}_{\text{pos}}} \bar{w}_{ij} \|\tilde{\mathbf{f}}_{i,\text{id}} - \mathbf{f}_{j,\text{id}}\|_2^2$$

$$\mathcal{L}_{\text{uniform}} = \frac{1}{2} \left(\log \left(\frac{1}{|\mathcal{I}_{\text{data}}|} \sum_{(i,j) \in \mathcal{I}_{\text{data}}} e^{-2\, d_F^2(\mathbf{f}_i, \mathbf{f}_j)} \right) + \log \left(\frac{1}{|\mathcal{I}_{\text{data}}|} \sum_{(i,j) \in \mathcal{I}_{\text{data}}} e^{-2\, d_F^2(\tilde{\mathbf{f}}_i, \tilde{\mathbf{f}}_j)} \right) \right)$$

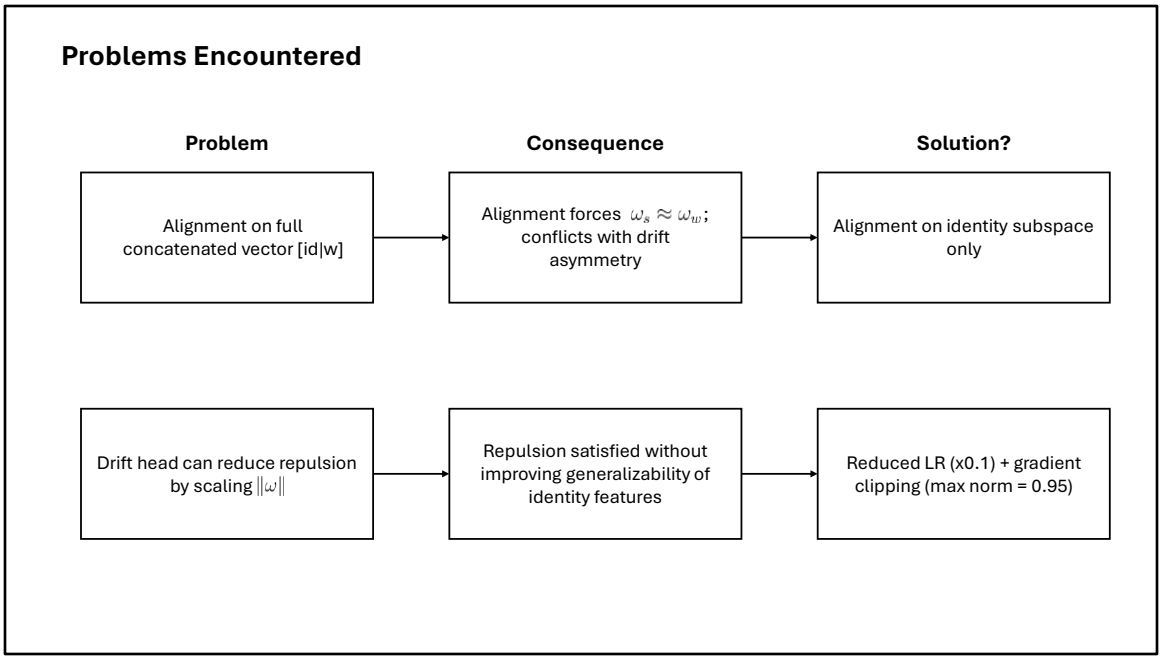
$$\mathcal{L}_{\text{domain}} = \frac{1}{2} \left(\log \left(\frac{\sum_i \sum_{j \in \mathcal{N}(\mathbf{f}_i)} e^{-2\, d_F^2(\mathbf{f}_i, \mathbf{c}_j)}}{\sum_i N} \right) + \log \left(\frac{\sum_i \sum_{j \in \mathcal{N}(\tilde{\mathbf{f}}_i)} e^{-2\, d_F^2(\tilde{\mathbf{f}}_i, \mathbf{c}_j)}}{\sum_i N} \right) \right)$$

Selective Application:
Alignment Loss is applied *only* to identity features. We force the *identity* to be invariant to augmentation, but we *allow* the drift to vary.
Uniformity & Domain Losses are applied to the full space to ensure proper distribution.

Key Insight: This setup mathematically decouples "invariance" (Identity) from "variance" (Drift), preventing them from canceling each other out.

Euclidean alignment among the omegas

- Why don’t we want uniform f and f tilde
- Look into disabling cosine similarity, maybe that would help



1. The Goal of Alignment (Invariance)
the **Alignment Loss** forces the embedding of a weakly augmented image (f_w (to be identical to the embedding of its strongly augmented version (f_s.(
Mathematically, it minimizes:

$$\| f_s - f_w \|^2 \to 0$$

If your embedding f contains a drift vector ω, this loss forces:

$$\omega_s \approx \omega_w$$

This means the network learns to output the **same drift vector** regardless of how the image is augmented (e.g., cropped, colored, erased).

2. The Goal of Finsler Drift (Asymmetry)
The purpose of the drift vector ω is to model **viewpoint asymmetry**.
Scenario: A UAV (drone) image looking down vs. a CCTV image looking straight.
The geometric distortion from UAV → CCTV is fundamentally different from CCTV → UAV.
Therefore, the drift ω_{UAV} must be different from ω_{CCTV} to capture this directional relationship.

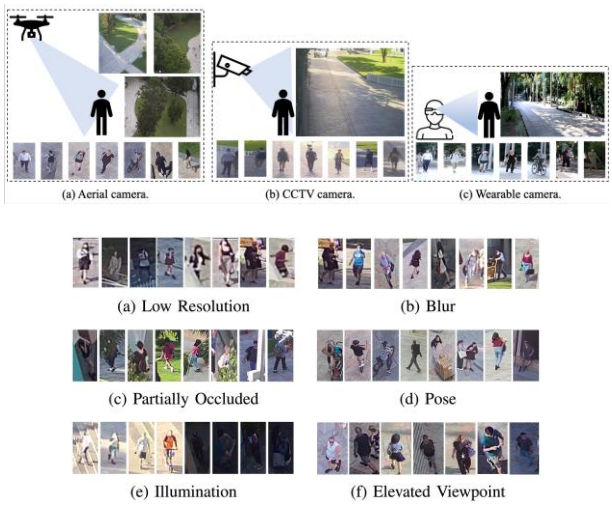
3. The Conflict

Deep learning augmentations (Random Crop, Random Erasing) simulate these viewpoint changes and occlusions.

If you enforce Alignment on ω: You force the network to say "The geometric bias of this image is the same, even if I crop out the background or change the aspect ratio." This effectively kills the drift mechanism, collapsing $\omega \to 0$ or a constant constant, reverting the space back to Euclidean geometry.

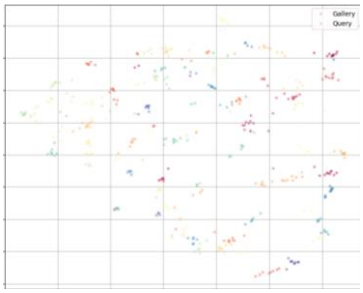
If you want useful Drift: The drift *must* be sensitive to the specific visual content (the specific crop/view). It *should* change when the view changes.

Dataset – AGReIDv2

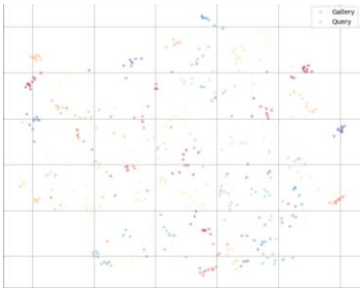


The Challenge: We are testing on AG-ReID v2, which includes Aerial (UAV), CCTV, and Wearable cameras.
Why it's Hard: The domain gap is extreme—matching a "top-down" drone view to a "eye-level" wearable view involves severe occlusion, resolution drops, and pose changes that symmetric models struggle to bridge.

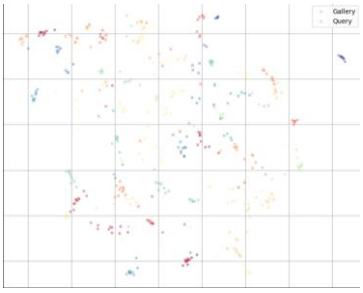
Visualization Modifications



BAU baseline



Finsler BAU – lam=1.5



Finsler BAU – lam=5.0

Visualization Modifications

BAU baseline | Finsler BAU – lam=1.5 | Finsler BAU – lam=5.0

Today

1. [**Model**]: Change the norm clamping of the omega norm to something smoother, e.g., log-barrier loss; sigmoid/exponential + stable behavior at zero?
2. [**Distance**]: Improve the geodesic approximation in the Finsler distance computation
3. [**Loss**]: Implement Euclidean alignment among the omega vectors
4. [**Loss**]: Compute another triplet loss using the inverse triplet (p, a, n'), where the positive is now the anchor (reverse direction) -> minimize the sum of these directional losses
5. [**Experiment**]: Reduce the embedding size of the drift vector
6. [**Master Thesis**]: Updates and Proposals

Pushed to Next Week

1. [**Sanity Test**]: Devise a small experiment to test the logic behind cross-res asymmetry and camera-viewpoint bias (e.g., small dataset + simple metric).
2. [**Loss**]: Consider implementing different repulsive forces to enforce domain generalization (other than uniformity and domain loss).
   - *Idea*: Something similar to the Finsler stress loss

- Finsler might not be classification, because classification wants to be invariant
- Preserving the drift
- Does any other downstream application of reid models that is not classification

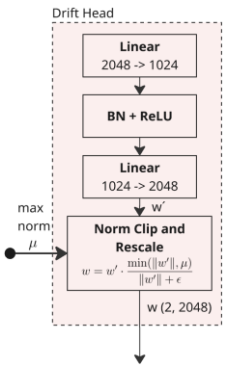
- Like training ResNet50 on ImageNet but we use it for something else

[Model] Improve Finsler Drift Head

Requirement: Model should match with Euclidean baseline when norm is zero
Suggestion: Avoid hard clamping to facilitate smooth gradients

Changes:

1. Initialized the last layer to zero
2. Removed the bias term in linear layers
3. Removed BatchNorm1d
4. Changed the clamping to something softer
   - Soft Tanh Scaling
   - Sigmoid Gated Scaling
   - Log-Barrier

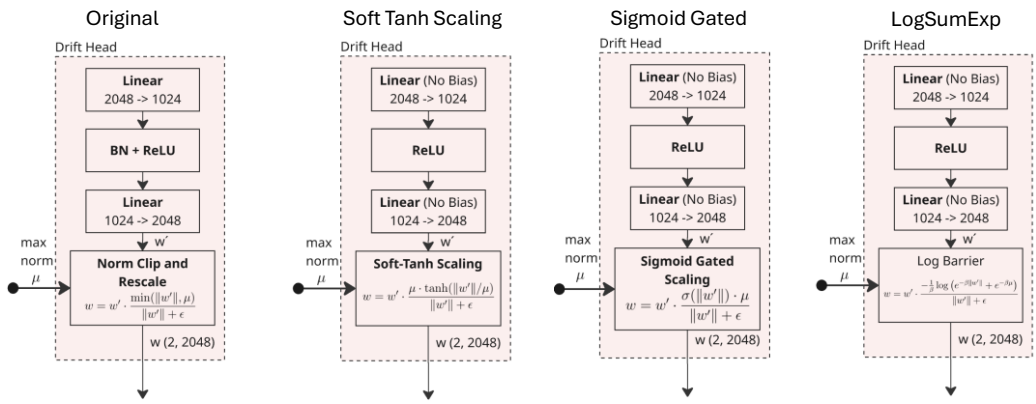


[Model] Improve Finsler Drift Head

Link to Interactive Graph

Changes:

1. Initialized the last layer to zero; Removed the bias term in linear layers + BN
2. Changed the clamping to something softer



Read up on He Initialization

- Option 1: hyperbolic tangent function to smoothly squash the norm of the drift vector so it asymptotically approaches max norm but never quite reaches it (behaves linearly for small norms).

2. Sigmoid Gated Scaling

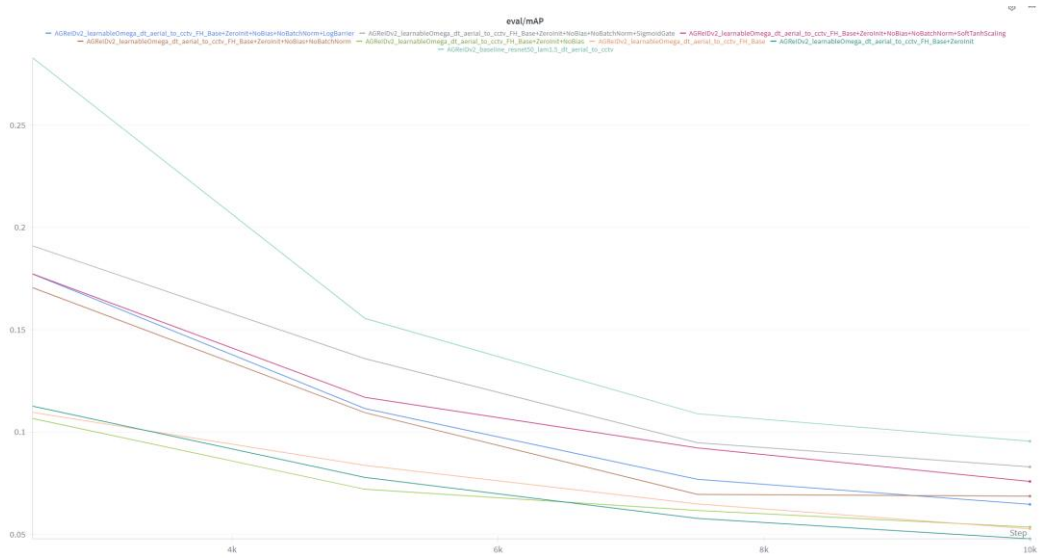
To allow the network to "decide" the magnitude more explicitly, we can gate the output. This is essentially an attention mechanism on the magnitude. Note that this doesn't strictly enforce max_norm unless you multiply by it, but it creates a smooth 0-to-max profile.

3. Log-Barrier / Soft-Maximum

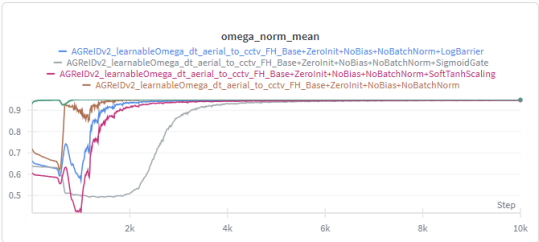
A "soft maximum" can be approximated using the LogSumExp trick or Softplus, but specifically for bounding a norm, a smooth minimum between the norm and max_norm is essentially what clamp does. Note that I found out that the mathematical expression here is wrong, hence the results shown soon are not faithful to the approach.

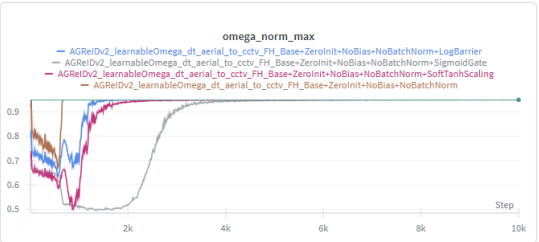
[Model] Improve Finsler Drift Head

[Model] Improve Finsler Drift Head



[Model] Improve Finsler Drift Head





[Distance] Improve the geodesic approximation in Finsler distance computation

We make two key simplifications:

- **Path Assumption**: Taking straight Euclidean line segment as the path between x and y
- **Field Assumption**: The drift vector field w changes linearly along this path

1 $$\gamma(t) = \mathbf{x}_{id} + t(\mathbf{y}_{id} - \mathbf{x}_{id}) \qquad \dot{\gamma}(t) = \mathbf{y}_{id} - \mathbf{x}_{id}$$

2 $$\mathcal{A}(\mathbf{x}, \mathbf{y}) = \int_0^1 \langle \omega(\gamma(t)), \dot{\gamma}(t) \rangle dt$$

3 $$\omega(\gamma(t)) \approx (1-t)\mathbf{x}_\omega + t\mathbf{y}_\omega \qquad \mathcal{A}(\mathbf{x}, \mathbf{y}) \approx \int_0^1 \langle (1-t)\mathbf{x}_\omega + t\mathbf{y}_\omega, \mathbf{y}_{id} - \mathbf{x}_{id} \rangle dt$$

$$\int_0^1 (1-t)dt = \frac{1}{2}, \quad \int_0^1 t dt = \frac{1}{2}$$

4 $$\mathcal{A}(\mathbf{x}, \mathbf{y}) = \langle \frac{\mathbf{x}_\omega + \mathbf{y}_\omega}{2}, \mathbf{y}_{id} - \mathbf{x}_{id} \rangle$$

$$\mathcal{A}(\mathbf{x}, \mathbf{y}) = \frac{1}{2}\left[(\mathbf{x}_\omega \cdot \mathbf{y}_{id}) - (\mathbf{x}_\omega \cdot \mathbf{x}_{id}) + (\mathbf{y}_\omega \cdot \mathbf{y}_{id}) - (\mathbf{y}_\omega \cdot \mathbf{x}_{id})\right]$$

Symmetrical Trapezoidal Approximation

1. **Define the Path**
   - We define the path $\gamma(t)$ in the identity feature space from x_{id} to y_{id} , parameterized by $t \in [0, 1]$.
   - The velocity vector (tangent to the path) is constant:

2. **Define the Integral**
   The asymmetric component $A(x, y)$ of the Randers metric is defined as the path integral of the drift one-form ω acting on the velocity vector.

3. **Linear Interpolation (Trapezoidal Approximation)**
   By assuming the drift vectors change linearly, we approximate $\omega(\gamma(t))$ as a weighted sum of the drift at start (x_ω) and end (y_ω).
   - Substituting this back into the integral
   - Since the velocity term is constant with respect to t, we integrate the scalar coefficients

4. **Step 4: The Closed Form Solution**
   - This simplifies the integral to the dot product of the **average drift vector** and the displacement vector.
   - Finally, we expand this dot product into the four specific terms

5. X and y have the same dimensions

Randers distance of the arc circle?

[Distance] Improve the geodesic approximation in Finsler distance computation

We make two key simplifications:

- **Path Assumption**: The geodesic is approximated as a straight Euclidean line segment
- **Field Assumption**: The drift vector field w changes linearly along this path

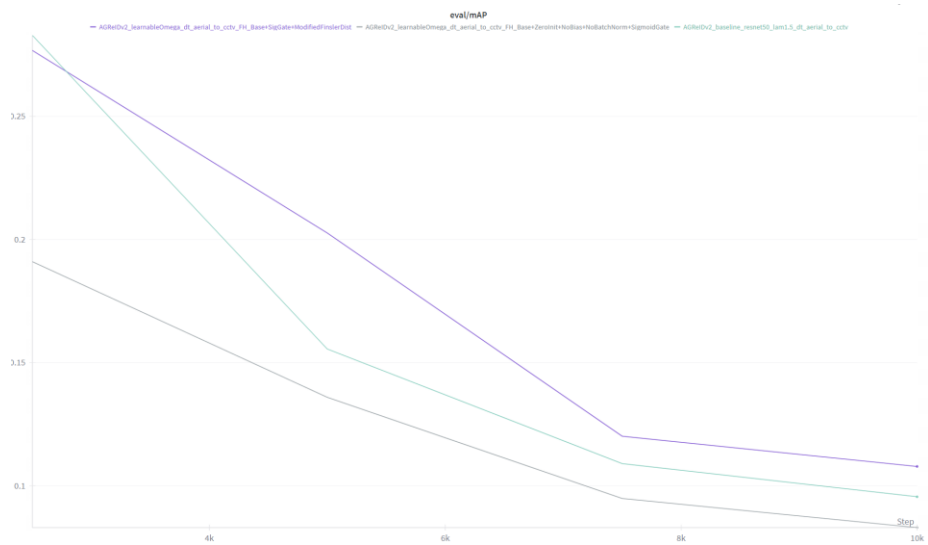
$$d_F(\mathbf{x}, \mathbf{y}) = \|\mathbf{x}_{\mathrm{id}} - \mathbf{y}_{\mathrm{id}}\|_2 + \langle \frac{\mathbf{x}_\omega + \mathbf{y}_\omega}{2}, \mathbf{y}_{id} - \mathbf{x}_{id} \rangle$$

$$d_F(\mathbf{x}, \mathbf{y}) = \|\mathbf{x}_{\mathrm{id}} - \mathbf{y}_{\mathrm{id}}\|_2 + \frac{1}{2}\left[(\mathbf{x}_\omega \cdot \mathbf{y}_{id}) - (\mathbf{x}_\omega \cdot \mathbf{x}_{id}) + (\mathbf{y}_\omega \cdot \mathbf{y}_{id}) - (\mathbf{y}_\omega \cdot \mathbf{x}_{id})\right]$$

[Distance] Improve the geodesic approximation in Finsler distance computation

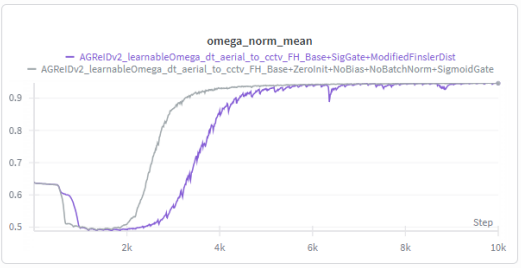
Do cross validation on the evaluation set (LOOCV)

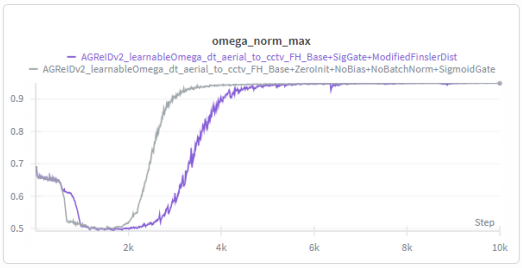
[Distance] Improve the geodesic approximation in Finsler distance computation



Do cross validation on the evaluation set (LOOCV)

[Distance] Improve the geodesic approximation in Finsler distance computation





Do cross validation on the evaluation set (LOOCV)

[Loss]: Implement Euclidean alignment among the omega vectors

Previously:

$$d_F(\mathbf{x}, \mathbf{y}) = \|\mathbf{x}_{\text{id}} - \mathbf{y}_{\text{id}}\|_2 + \mathbf{x}_\omega \cdot (\mathbf{y}_{\text{id}} - \mathbf{x}_{\text{id}})$$

$$\mathcal{L}_{\text{align}} = \sum_{(i,j)\in\mathcal{I}_{\text{pos}}} \bar{w}_{ij} \|\tilde{\mathbf{f}}_{i,\text{id}} - \mathbf{f}_{j,\text{id}}\|_2^2$$

Now:

$$\mathcal{L}_{\text{align}} = \sum_{(i,j)\in\mathcal{I}_{\text{pos}}} \bar{w}_{ij} \|\tilde{\mathbf{f}}_{i,\text{id}} - \mathbf{f}_{j,\text{id}}\|_2^2 + \sum_{(i,j)\in\mathcal{I}_{\text{pos}}} \bar{w}_{ij} \|\tilde{\mathbf{f}}_{i,\omega} - \mathbf{f}_{j,\omega}\|_2^2$$

1. When Drift Represents "Static" Domain Characteristics

If the drift vector (w) is intended to capture the inherent style of the camera or scene (e.g., lighting, background, camera angle) rather than the specific noise added by data augmentation, aligning it forces the model to produce the **same style embedding** regardless of whether the input image is weakly or strongly augmented.

- **Without Drift Alignment:** The model might encode the strong augmentation (e.g., heavy color jitter or cutout) *into* the drift vector. This means $w_{strong} \neq w_{weak}$.
- **With Drift Alignment:** You force $w_{strong} \approx w_{weak}$ .This ensures the drift vector purely captures the underlying domain style (e.g., "This is Camera 1") and ignores the transient augmentation noise.

2. To Prevent Information Leakage

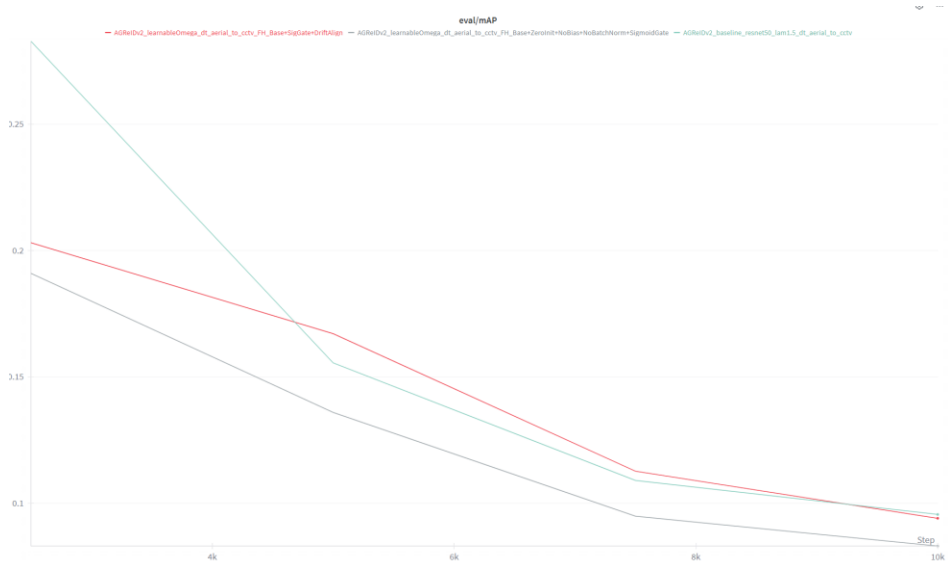
If the drift vector is allowed to change freely between augmentations, the network might "hide" identity information inside the drift vector to satisfy the reconstruction or classification losses easier, rather than disentangling them properly. Forcing consistency (alignment) acts as a regularizer, making the decomposition into $Identity + Drift$ more robust.

3. In Contrast to "Drift as Noise"

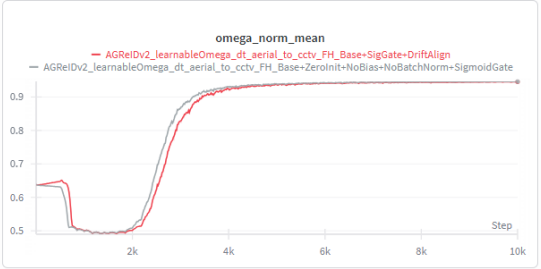
You would **NOT** align the drift vector if you explicitly want the drift component to capture the augmentation itself (i.e., if you treat the augmentation as a domain shift). In that case, w_{strong} *should* be different from w_{weak} .However, in most ReID contexts involving "f_w" (weak) and "f_s" (strong), you usually want the features (both identity and global style) to remain consistent for the same image instance

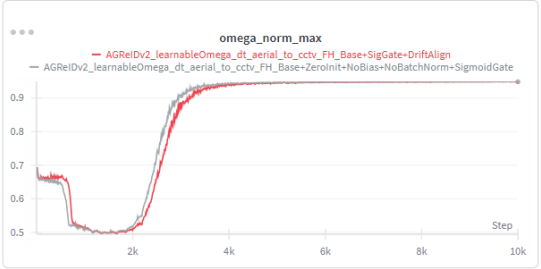
[Loss]: Implement Euclidean alignment among the omega vectors

[Loss]: Implement Euclidean alignment among the omega vectors



[Loss]: Implement Euclidean alignment among the omega vectors





[Loss]: Implement reverse triplet loss

Hypothesis: Minimizing the sum of these directional losses shapes the embedding space such that the drift "w" minimizes the retrieval error in both directions independently.

Before

$$\mathcal{L}_{\text{triplet,BH}} = \frac{1}{B}\sum_{i=1}^{B} \max\left(0, \max_{p\in\mathcal{P}(i)}\{d_f(x_i, x_p)\} - \min_{n\in\mathcal{N}(i)}\{d_F(x_i, x_n)\} + \alpha\right)$$

Now

$$\mathcal{L}_{\text{bi-triplet}} = \frac{1}{2}\left(\mathcal{L}_{\text{tri,f}}(i) + \mathcal{L}_{\text{tri.b}}(i)\right)$$

$$\mathcal{L}_{\text{tri,f}}(i) = \frac{1}{B}\sum_{j=1}^{B} \max\left(0, \max_{p\in\mathcal{P}(j)}\{d_F(x_j, x_p)\} - \min_{n\in\mathcal{N}(j)}\{d_F(x_j, x_n)\} + m\right)$$

$$\mathcal{L}_{\text{tri,b}}(i) = \frac{1}{B}\sum_{j=1}^{B} \max\left(0, \max_{p\in\mathcal{P}(j)}\{d_F(x_p, x_j)\} - \min_{n\in\mathcal{N}(j)}\{d_F(x_n, x_j)\} + m\right)$$

Intuition: Since $d(a, b) \neq d(b, a)$,optimizing $L(a, p, n)$ only ensures that a can retrieve p. It guarantees nothing about p retrieving a. Minimizing the sum ensures the drift component facilitates retrieval in *both* directions. This aligns with the hypothesis about shaping the drift w.

Standard Forward Loss (mat_dist):

We iterate through each row i (treating i as the **anchor**).
We search the columns to find a positive p and negative n.
We minimize: $d(x_i, x_p) - d(x_i, x_n)$.
Meaning: "Make anchor i closer to its hardest positive neighbor than to its nearest impostor."

Inverse Loss (mat_dist.t()):

We transpose the matrix, so row j now contains distances *from* all other points *to* j: $[d, (x_0, x_j), d(x_1, x_j) \ldots]$.
We iterate through each row j of this transposed matrix. Here, j **is the target**, and the "columns" (which were originally rows) are the potential

queries.

_batch_hard selects the index q (query) that maximizes $d(x_q, x_j)$ among positives, and index k that minimizes $d(x_k, x_j)$ among negatives.

Meaning: "Make target j be retrieved by its 'worst' aligned positive query q better than it is retrieved by its 'best' aligned negative query k."

Why this distinction matters

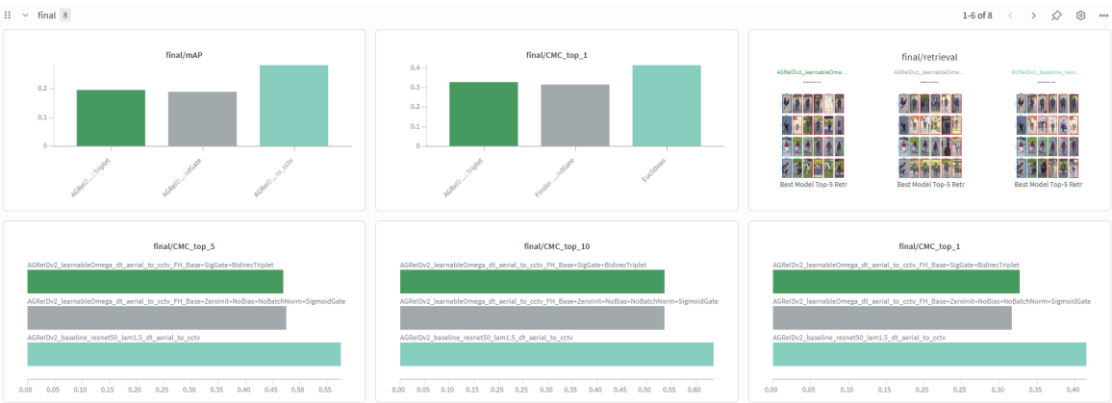
In a symmetric Euclidean space, $d(x_q, x_j) = d(x_j, x_q)$ .The set of distances is identical, but the *mining* might select different indices.

In the forward pass, for anchor A, maybe B is the hardest positive ($d(A, B)$ is max).

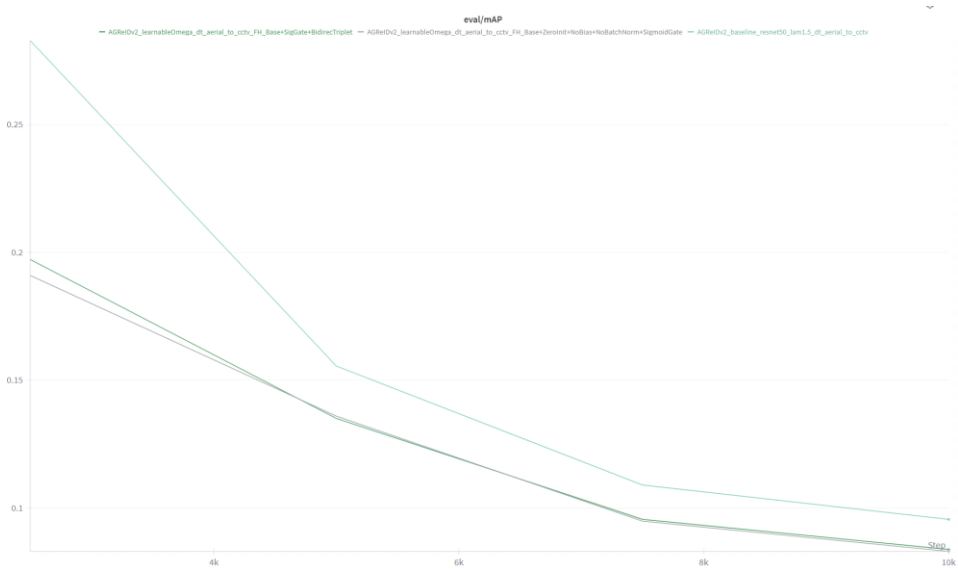
In the inverse pass, for target A, maybe C is the hardest query ($d(C, A)$ is max).

- Look into the learning rate scheduling, should decrease instead of increase
- Inside the parentheses cannot be the same running variable as in the sum

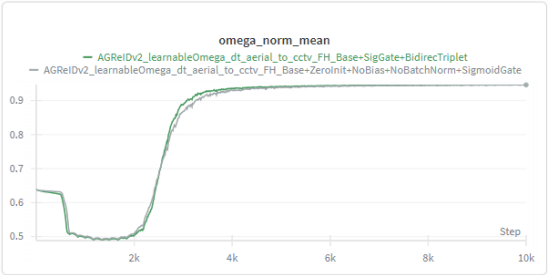
[Loss]: Implement reverse triplet loss

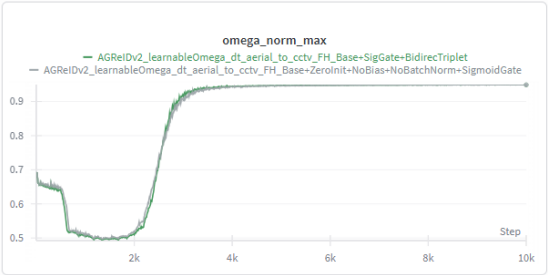


[Loss]: Implement reverse triplet loss



[Loss]: Implement reverse triplet loss





[Experiment] Reduce the embedding size of the drift vector

Previously:

$$d_F(\mathbf{x}, \mathbf{y}) = \underbrace{\|\mathbf{x}_{\text{id}} - \mathbf{y}_{\text{id}}\|_2}_{\text{identity distance}} + \underbrace{\mathbf{x}_\omega \cdot (\mathbf{y}_{\text{id}} - \mathbf{x}_{\text{id}})}_{\text{viewpoint-dependent drift}}$$

Now:

$$d_F(\mathbf{x}, \mathbf{y}) = \|\mathbf{x}_{\text{id}} - \mathbf{y}_{\text{id}}\|_2 + \sum_{k=1}^{S} \mathbf{x}_\omega^{(k)} (\mathbf{y}_{\text{id}}^{(k)} - \mathbf{x}_{\text{id}}^{(k)})$$

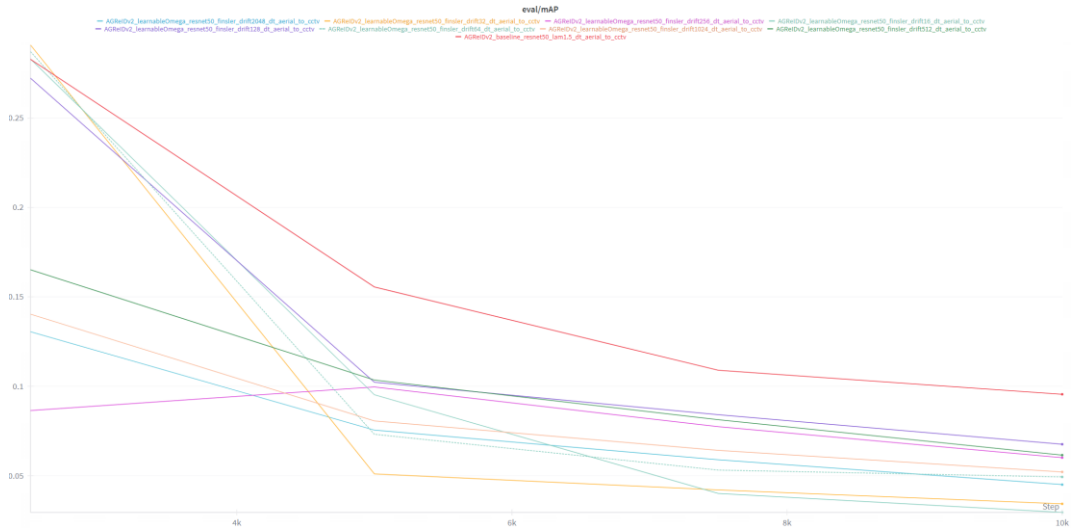
$$S = \min(\dim(\mathbf{x}_{\text{id}}), \dim(\mathbf{x}_\omega))$$

Note that this experiment was not made in tandem with the previous. It was done as the first one, as I didn't know how long the sweep was going to take

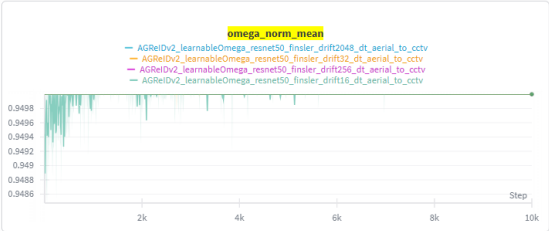
Multiply the sum by N/S, where N is the embedding size of x and y

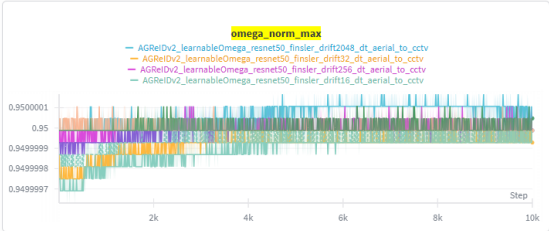
[Experiment] Reduce the embedding size of the drift vector

[Experiment] Reduce the embedding size of the drift vector



[Experiment] Reduce the embedding size of the drift vector





[Master Thesis] Updates and Proposal

Master's Thesis Research Proposal

Target Group: Computer Vision Group (CVG), TUM
Date: February 18, 2026

Candidate: Lee Zhuo Le (03795610)
Program: M.Sc. Robotics, Cognition, Intelligence

Industrial Collaboration:
Isarsoft GmbH

Research Topic & Motivation

Proposed Title: Beyond Cosine: Geometric Metrics for Vision Foundation Model Embeddings

Problem Statement: While vision foundation models map images to $\mathbb{R}^d$, the standard practice of using cosine similarity implicitly assumes a flat Euclidean geometry. Recent studies show that while scaling laws and architectural optimizations like QK-Normalization have largely mitigated naive artifacts such as anisotropy [1] and "hubness" [2] in modern models [3], Euclidean metrics can limit downstream tasks. Explicit non-Euclidean metric modeling is required for tasks exhibiting asymmetric relationships and hierarchical logic [4], or multi-view spatial consistency [5]. Furthermore, current geometric assumptions

Next Week

1. [**Training**]: Fix the learning rate scheduler such that it anneals over time instead of increasing; redo all the experiments
2. [**Experiment**]: Combine all of the improvements in this week into one model, going step-by-step to understand the contribution of each modification
   - Do cross-validation on the evaluation/test set to rule out random seeding behaviors
3. [**Model**]: Fix the LogSumExp activation function / approximation and re-run the test
4. [**Theory/Experiment**]: Brainstorm a better approximation of the Rander's drift vector compared to the symmetric trapezoidal approximation
5. [**Theory/Experiment**]: Consider the improvements in reducing the drift vector dimension as a potential scaling problem (multiply by N/S) – substantiate this argument and test it out
6. [**Sanity Test**]: Devise a small experiment to test the logic behind cross-res asymmetry and camera-viewpoint bias (e.g., small dataset + simple metric).
7. [**Loss**]: Consider implementing different repulsive forces to enforce domain generalization (other than uniformity and domain loss).
   - *Idea*: Something similar to the Finsler stress loss